Enhanced Smart Contract Vulnerability Detection via Graph Neural Networks: Achieving High Accuracy and Efficiency

Chang Xu, Huaiyu Xu, Liehuang Zhu, *Senior Member, IEEE*, Xiaodong Shen, and Kashif Sharif, *Senior Member, IEEE*

***Abstract*—As blockchain technology becomes prevalent, smart contracts have shown significant utility in finance and supply chain management. However, vulnerabilities in smart contracts pose serious threats to blockchain security, leading to substantial economic losses. Therefore, developing effective vulnerability detection solutions is urgent. To address this issue, we propose a method for detecting vulnerabilities in smart contracts using graph neural networks (GNNs) that can identify eight common vulnerabilities. Our method is fully automated, applicable to all Ethereum smart contracts, and does not require expert-defined rules or manually defined features. We extract the Control Flow Graph and Abstract Syntax Graph from the smart contract code, which are then processed by a GNN to generate feature vectors for classification. Experiments on a real Ethereum dataset demonstrate that our method significantly outperforms existing state-of-the-art approaches. For individual detection tasks, the combined source code and bytecode method achieves an average accuracy of 95.78%, with a peak of 99.13%, and an average F1 score of 93.80%. Compared to competitors, our method shows an average improvement of 51.92% in accuracy and 47.21% in F1 score. The bytecode-only method achieves an average accuracy of 94.68% and an F1 score of 92.36%. For multi-class tasks, both methods achieve high accuracies of 91.26% and 87.34%, with F1 scores of 97.42% and 96.43%, respectively.**

***Index Terms*—Smart contract, Ethereum, blockchain, vulnerability detection, graph neural network**

I. Introduction

ETHEREUM is the most active blockchain platform, supported by an extensive user and developer community, with a market capitalization of $450 billion [1]. Smart contracts [2], autonomously executed protocols on the platform, embed rules directly into code, enabling decentralized applications (DApps). Developers use code to manage digital assets [3]. Like other codebases, smart contracts are prone to vulnerabilities [4]. Flaws in logic, programming, and toolchains lead to issues such as integer overflow [5], reentrancy [6], call injection [7] (or code injection), and access control flaws [8]. Due to Ethereum's growing economic value, attackers increasingly target it. Large-scale attacks, such as DAO [9] and Parity [10], have led to millions to billions of dollars in financial losses.

Only about one-third of contracts on the Ethereum blockchain have publicly disclosed source code, while the rest exist solely as EVM bytecode [11]. Contracts with disclosed source code are more vulnerable to attack due to their semantic transparency [12], [13]. Most existing work focuses on bytecode analysis [14], [15], but bytecode lacks the ability to describe concepts like structs and inheritance, unlike the high-level language Solidity, potentially affecting detection accuracy [16]. Therefore, we focus on analyzing contracts with source code available on-chain.

Current analysis tools primarily use static or fuzz testing methods [12], [17], [18], [19]. These approaches rely on specific rules defined by human experts to detect vulnerabilities [20]. They verify if the contract logic matches predefined patterns. However, since not all vulnerabilities—especially complex ones with multiple triggering conditions—can be precisely defined, this limits the scalability and accuracy of detection methods. Moreover, the reliance on expert knowledge results in high labor costs [21], [22].

A few studies have used machine learning to detect smart contract vulnerabilities [22], [23]. These tools, benefiting from automated data processing, have shown improved detection performance. However, most of these methods are limited to source code and do not work effectively with bytecode-only contracts [22], [24]. Additionally, most deep learning approaches only support binary classification, identifying the presence or absence of specific vulnerabilities [11], [25], [26]. Effective methods for detecting multiple types of vulnerabilities simultaneously still need to be improved.

We present a graph neural network (GNN) detection method that learns vulnerabilities during training, eliminating the need for additional rules or human intervention. Experimental results demonstrate that our method achieves faster detection speeds

Received 20 November 2024; revised 28 April 2025; accepted 12 May 2025. Date of publication 15 May 2025; date of current version 16 June 2025. This research was supported by the National Natural Science Foundation of China under Grant 62272042. Recommended by T. Menzies. *(Corresponding author: Chang Xu.)*

Chang Xu, Huaiyu Xu, Liehuang Zhu, and Xiaodong Shen are with the School of Cyberspace Science and Technology, Beijing Institute of Technology, Beijing 100811, China (e-mail: xuchang@bit.edu.cn; xuhuaiyu@bit.edu.cn; liehuangz@bit.edu.cn; shenxiaodong@bit.edu.cn).

Kashif Sharif is with the School of Computer Science and Technology, Beijing Institute of Technology, Beijing 100811, China (e-mail: kashif@bit.edu.cn).

Digital Object Identifier 10.1109/TSE.2025.3570421

0098-5589 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Consortium - Saudi Arabia SDL. Downloaded on February 23,2026 at 18:39:25 UTC from IEEE Xplore. Restrictions apply.

than existing static solutions. Moreover, unlike many existing tools that detect only a single vulnerability, our approach can detect multiple vulnerabilities simultaneously.

The key contributions of our work are as follows:

1) We introduce an engine that transforms smart contracts into feature vectors. It comprises two components: an Abstract Syntax Graph (ASG) built from the Solidity source code, and a Control Flow Graph (CFG) constructed from the contract's bytecode. These features are then merged to produce a high-dimensional vector.
2) We present a smart contract vulnerability detection method named Bytecode and Source code Graph Neural Networks Vulnerability Detector (BSGVD). It takes both source code and bytecode as inputs, using supervised learning to train a graph neural network to detect eight common vulnerabilities. We labeled 25,350 contracts with Slither, forming a dataset of benign and vulnerable contracts.
3) We also develop Bytecode Graph Neural Networks Vulnerability Detector (BGVD) to detect vulnerabilities solely from EVM bytecode. While BGVD may not be as effective as BSGVD, its performance surpasses most existing tools.
4) We split the dataset into training and validation sets, conducting ten-fold cross-validation experiments. BSGVD shows outstanding performance, achieving detection accuracies above 94% for individual vulnerabilities, with an average accuracy of 95.78% and an F1 score of 93.80%, surpassing most existing tools. Similarly, BGVD achieves an average accuracy of 94.68% and an F1 score of 92.36%.
5) Our tools can perform both binary classification for single vulnerabilities and mixed detection for multiple vulnerabilities. In the same dataset, BSGVD achieves an average accuracy of 91.26% and an F1 score of 97.42%, while BGVD achieves 87.34% accuracy and an F1 score of 96.43%. Both methods provide average improvements over competitors, with accuracy improvements of 51.92% and 51.24%, and F1 score improvements of 47.21% and 45.73%, respectively.

The remainder of this paper is organized as follows. Section II reviews the related work. Section III provides foundational background. Section IV describes the implementation of BSGVD and BGVD. Section V presents experimental evaluation and analysis of our approach. Finally, Section VI concludes the paper.

II. Related Works

Existing smart contract vulnerability detection techniques are primarily classified into static and dynamic analysis. Many static analysis methods rely on symbolic execution or intermediate representation-based approaches, including Oyente [12], Mythril [17], Osiris [18], Securify [27], Sereum [28], eThor [29], and SmartCheck [30].

These methods rely on human experts defining rules for vulnerabilities, which are then used for detection [31]. During detection, all feasible paths are simulated to check adherence to the rules, resulting in high accuracy, as seen with Slither [32], which we use in this paper. However, this rule-based approach struggles to detect new vulnerabilities, as each new vulnerability must be generalized into machine-understandable rules, which is a challenging task. Simulating all paths is computationally expensive, especially as contract code size increases and the number of executable paths grows rapidly [11]. This makes detecting vulnerabilities in large contracts both time and resource-intensive. Experimental results in this paper show that static detection may underperform for specific vulnerabilities, particularly when defining precise rules or handling multiple triggering conditions [16], [22].

Fuzz testing-based detection tools, such as ContractFuzzer [19] and Confuzzius [33], have received comparatively less attention in the research community. Unlike symbolic execution, fuzz testing is better suited for analyzing large contracts, but it is cumbersome and inherently random, making it less effective for large-scale, precise vulnerability detection.

With the rapid advancement of machine learning, numerous works combining machine learning with security analysis have emerged, including code clone detection [34], vulnerability detection [16], and code search [35]. Recent work also utilizes large models for blockchain security analysis, such as GPTScan [36], and has shown promising results. Machine learning is increasingly being applied to smart contract vulnerability detection. For instance, [37] combines machine learning with manual feature extraction using sentence encoders. Additionally, machine learning has been integrated into dynamic fuzzers, as seen in [38]. Fully automated methods have also been proposed, such as AME [22], CGE [39], TMP [24], DR_GCN [24], and ReChecker [23].

III. Preliminaries

This section introduces the key concepts and techniques used in our method.

A. Solidity Source Code and Bytecode

Smart contracts on the Ethereum blockchain are typically written in Solidity [40], which contains the core logic and functionality defined by developers. Compilers then translate the Solidity code into bytecode, representing the contract in binary form (Fig. 1). Bytecode is a sequence of low-level instructions executed by the Ethereum Virtual Machine (EVM) and expressed as a string of hexadecimal values. It defines the operations and behaviors of the smart contract. Once deployed, the contract's bytecode is permanently stored on the Ethereum blockchain and executed by miner nodes [41].

B. Basic Block and Control Flow Graph

A basic block is a continuous code fragment with a single entry and exit point. The entry is the first instruction, while the exit is the last or a branching instruction. In a Control Flow Graph (CFG) [42], basic blocks are nodes, and control flow transitions are edges. CFG analysis helps understand program execution paths and dependencies between code blocks, enhancing comprehension of the program's structure and behavior [43].

Authorized licensed use limited to: Consortium - Saudi Arabia SDL. Downloaded on February 23,2026 at 18:39:25 UTC from IEEE Xplore. Restrictions apply.

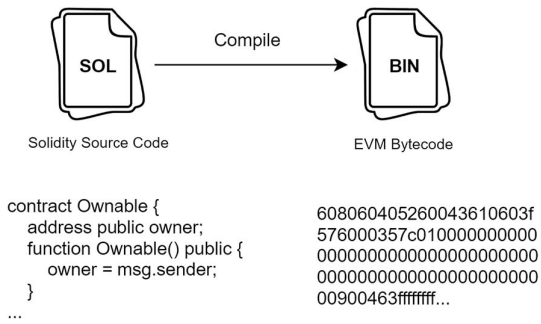

Fig. 1. Smart contract source code and bytecode.

The EVM CFG Builder [44] is a tool for extracting CFG from EVM bytecode. It is widely used in tools like Ethersplay, Manticore, and Trail of Bit, demonstrating its maturity in the field.

C. Abstract Syntax Tree and Abstract Syntax Graph

An Abstract Syntax Tree (AST) [45] is an abstract representation of source code that describes the program's syntactic structure in a tree-like format. Each node represents a syntactic element, such as expressions, statements, or function definitions. AST abstracts away specific code details, making the syntactic structure more straightforward to understand.

We extend AST to an Abstract Syntax Graph (ASG) [46], which includes additional semantic and control flow information. ASG represents semantic relationships and control flow by transforming nodes and adding edges, enhancing modeling and analysis accuracy.

D. Smart Contract Vulnerabilities

Smart contract vulnerabilities are common, and numerous attacks exploit them. We focus on eight common vulnerabilities: controlled-delegatecall, controlled-array-length, unchecked-transfer, erc20-interface, locked-ether, shadowing-abstract, constant-function-asm, and reentrancy-eth [32], [47]. Table I provides detailed descriptions of these vulnerabilities, along with their Smart Contract Weakness Classification (SWC) numbers, which readers can refer to on the official SWC website[1].

E. Slither and FastText

We use Slither [32] for supervised learning, employing it for static analysis of Ethereum smart contracts. Slither aims to discover security vulnerabilities and potential issues, and as of the writting of this paper (April, 2024), it continues to receive updates. It identifies vulnerabilities through bytecode analysis and provides detailed reports.

We also use fastText [48] for word-to-vector conversion. Developed by Facebook AI Research, fastText is an open-source library for text classification and vectorization, providing a rapid and efficient method for text processing. In this paper, we utilize the pre-trained model without additional training.

[1]SWC website: https://swcregistry.io/

F. Graph Neural Networks

We draw inspiration from the Saliency-Aware Regularized Graph Neural Network (SARGNN) [49], which uses a traditional graph neural network as a backbone to learn node features. SARGNN employs Graph Neural Memory (GNM) to derive compact graph representations, estimating global node saliency through the similarity between the compact graph representation and node features. This saliency is then used to regularize backbone aggregation, facilitating efficient message passing and node feature aggregation. SARGNN has demonstrated excellent performance compared to other GNNs.

IV. Proposed Scheme

Designing a solution with graph neural networks involves several key stages. First, the smart contract code data is converted into a graph format, defining the meanings of nodes and edges. Next, an appropriate graph neural network model is selected according to the characteristics of smart contract cloning, and the network structure is designed. Then, training and validation sets are constructed to tune the graph neural network model.

A. Overview

The overall architecture is presented in Fig. 2. First, a dataset containing the bytecode and source code of contracts is obtained from the Ethereum blockchain. Slither is then used to label the contracts, identifying the presence and type of vulnerabilities. CFGs are derived from the bytecode, while ASGs are extracted from the source code (IV-B). We use fastText to convert the nodes in both graphs into vectors (IV-C). Subsequently, graph neural networks transform the graphs into vector representations. Feature fusion is then performed to obtain the final vector representation of the contract (IV-D). Finally, the fused embeddings are input into a classification neural network to detect vulnerabilities and specify their types (IV-E). Details regarding the design and training process of the graph neural network are detailed in sections IV-D and IV-E.

B. Graph Extraction

The process of extracting graphs from code consists of two primary steps. The first step is control flow graph extraction, which captures control flow relationships from the program's bytecode. The second step is abstract syntax graph extraction, which supplements data flow and structural dependency relationships from the program's source code.

1) Control Flow Graph Extraction: We use the EVM CFG Builder to extract the CFG from the bytecode. First, the smart contract bytecode is disassembled into a more readable sequence of opcodes. Next, the opcodes are divided into basic blocks, where each basic block consists of a continuous series of instructions up to a branch or jump instruction. We then analyze the opcodes to detect conditional statements, loops, and jump instructions, capturing the control flow information of the code. Finally, the CFG is constructed based on the relationships between these basic blocks and the overall control flow structure.

Authorized licensed use limited to: Consortium - Saudi Arabia SDL. Downloaded on February 23,2026 at 18:39:25 UTC from IEEE Xplore. Restrictions apply.

TABLE I
THE EIGHT VULNERABILITIES IN OUR DATASET

Vulnerabilities	Details	Count
controlled-delegatecall (SWC-112)	Delegatecall or callcode functions to external address.	778
reentrancy-eth (SWC-107)	Before the state variable is changed for the first time, malicious contracts callback to the calling contract. This can lead to unexpected interactions between different function calls.	1,015
controlled-array-length	Functions that enable setting the length of an array directly.	1,900
unchecked-transfer	The return value is not checked for an external 'transfer'/'transferFrom' call.	6,219
erc20-interface	The return value of the ERC20 function is incorrect.	5,950
locked-ether	Contract that can receive payments but lacks withdrawal functionality.	6,658
shadowing-abstract	State variables shadowing from abstract contracts.	1,505
constant-function-asm	Declare functions as constant/immutable/view using assembly code.	1,325

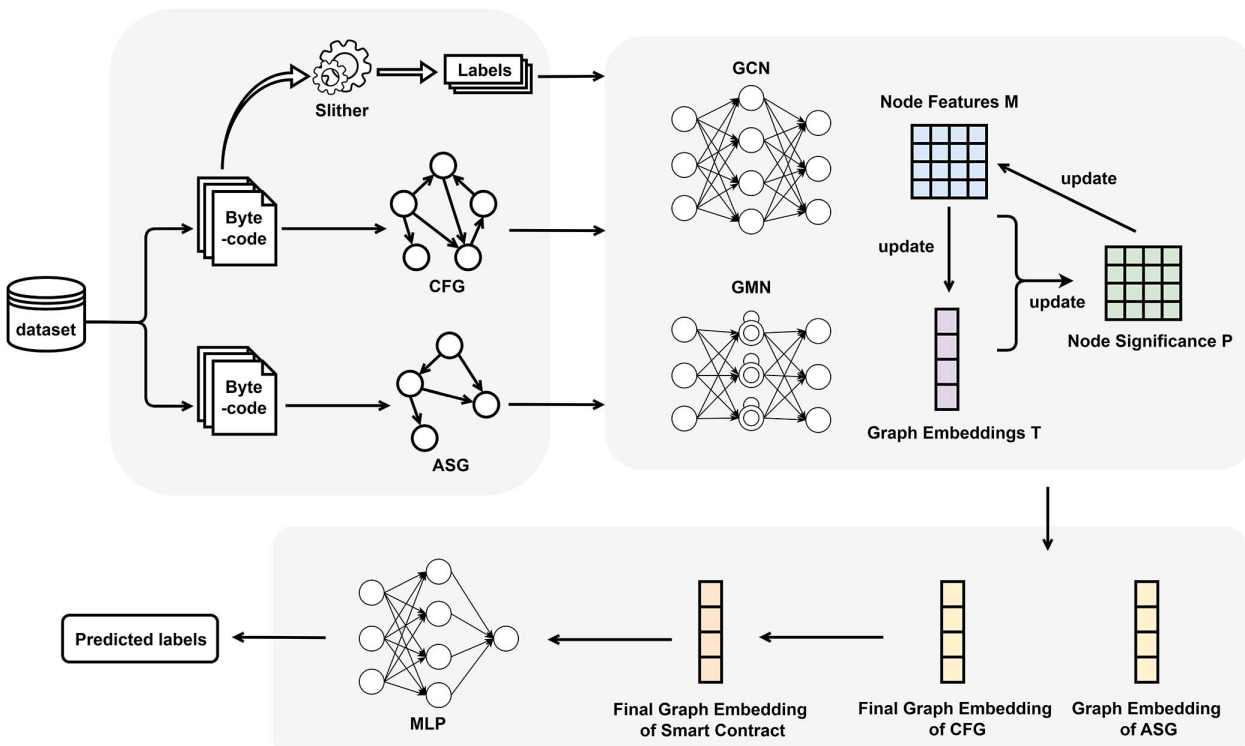

Fig. 2. High-level overview of our smart contract vulnerability detection system using graph neural networks. (1) Label contracts in the dataset using slither. (2) Generate an ASG from the contract's source code and a CFG from the bytecode. (3) Train a graph neural network with supervised learning, where node features Mupdate the graph embedding T. Calculate node significance P to guide feature updates in the next layer. (4) Fuse ASG and CFG embeddings and input them into a multi-layer perceptron for classification.

The CFG is denoted as $G_{cfg} = (V, E)$, where V represents the set of nodes and E denotes the set of edges. Each node $v_i \in V$ represents a basic block, and each edge $e_{(i,j)} \in E$ represents the control flow dependency between nodes v_i and v_j.

2) Abstract Syntax Graph Extraction: The extraction of the Abstract Syntax Graph from the source code in this paper draws inspiration from the approach in [49]. First, the Solidity compiler (solc) compiles the contract source code, generating an AST to depict the contract code's syntactic structure accurately. Then, relationships such as node access, function calls, conditional judgments, and other dependencies related to control and data flow are established, enriching the graph representation's meaning. This results in a more accurate and comprehensive representation of the code.

The ASG is represented as $G_{asg} = (V, E)$, where V represents the set of nodes, and E represents the set of edges. Each node $v_i \in V$ corresponds to a statement in the source code, and each edge $e_{(i,j)} \in E$ represents control flow, data flow, or data structure dependencies from node v_i to v_j.

Control flow reflects the program's execution order. In the source code of smart contracts, constructs such as *for*, *while*, and *if* control program flow. In the AST, these control nodes

Authorized licensed use limited to: Consortium - Saudi Arabia SDL. Downloaded on February 23,2026 at 18:39:25 UTC from IEEE Xplore. Restrictions apply.

can be identified by matching their tokens to corresponding statements. Constructing sequential execution edges improves the representation of control flow by processing these keywords. Data flow reflects the order of data transmission within the program. The positions of variables in the AST are identifiable, allowing the data flow relationships between variables to be established based on the execution order of code statements.

C. Node Embedding Generation

The contract graphs are converted into inputs suitable for graph neural networks in this phase. The *datasets'* data structure from *torch_geometric* is used to store the graphs. For CFG and ASG, nodes are treated as text blocks and then vectorized using fastText. As a result, each node acquires a feature vector of dimension d_M. This produces a node feature matrix $M \in \mathbb{R}^{N \times d_M}$, where N represents the number of nodes. The graph edges are represented using a sparse adjacency matrix.

D. Smart Contract Embedding Generation

We build a GNN architecture based on the SARGNN model [49]. First, the saliency of nodes in the graph is learned to guide neighborhood aggregation within the GNN. By promoting the propagation of features from nodes with high saliency, the accuracy of the graph representation is enhanced. The GNN utilized in this paper consists of two core components that complement each other to enhance the graph representation gradually. The traditional GCN network serves as the base network, while the Graph Neural Memory extracts the graph's embedding from the node representations of the base network.

1) Node Characteristic Distillation: For the input graph G, the Graph Neural Memory extracts node features from the GCN, resulting in graph embeddings:

$$T^l = F_T(M^{l-1}, T^{l-1}). \tag{1}$$

The graph representation T^l at layer l is obtained from the node feature matrix M^{l-1} and the graph embedding T^{l-1} at layer $l-1$. All graph embeddings are d_T-dimensional vectors. The initial graph embedding, T^0, is randomly initialized. The update function F_T of the Graph Neural Memory is modeled as follows:

$$\begin{aligned} &U_T = T^{l-1} W_T^U, \quad X_T = M^{l-1} W_T^X, \quad Y_T = M^{l-1} W_T^Y, \\ &T^l = F_{\text{MLP}}(T'), \quad T' = \text{softmax}\left(\frac{U_T \mathbf{X}_T^\top}{\sqrt{d}}\right) Y_T. \end{aligned} \tag{2}$$

Here, $W_T^U \in \mathbb{R}^{d_T \times d}$, $W_T^X \in \mathbb{R}^{d_M \times d}$, and $W_T^Y \in \mathbb{R}^{d_M \times d}$ are learnable parameter matrices. In each layer of the Graph Neural Memory, the graph embedding is updated by attending to the node features from the previous layer in the base network and performing a cross-attention operation for feature distillation. The module F_{MLP}, a Multilayer Perceptron (MLP), consists of two fully connected layers with transformation matrices $W_{\text{MLP}}^a \in \mathbb{R}^{d_T \times d}$ and $W_{\text{MLP}}^b \in \mathbb{R}^{d \times d}$, with an activation function applied between them. To avoid data explosion resulting from dot product operations, a scaling factor of $\frac{1}{\sqrt{d}}$ is used.

2) Node Significance Evaluation: The updated graph embedding is further used to evaluate node significance, which aids in filtering nodes that are more important for global aggregation in the GCN. Node significance within the graph is evaluated using the function F_P:

$$P^{l-1} = F_P(T^l, M^{l-1}). \tag{3}$$

In Equation (3), the significance of nodes $P^{l-1} \in \mathbb{R}^N$ in layer $l-1$ is determined by both the graph embeddings T^l and the node features M^{l-1}. The sum of all elements in P^{l-1} is 1. Node significance illustrates the coupling between individual node and the global graph representation. The update function F_P for node significance is modeled as:

$$\begin{aligned} U_P &= T^l W_P^U, \quad X_P = M^{l-1} W_P^X, \\ P^{l-1} &= \text{softmax}\left(\frac{U_P \mathbf{X}_P^\top}{\sqrt{d}}\right). \end{aligned} \tag{4}$$

Here, $W_P^U \in \mathbb{R}^{d_T \times d}$ and $W_P^X \in \mathbb{R}^{d_M \times d}$ are learnable parameter matrices.

3) Network Regularization: Using the node significance P^{l-1} for regularization, the base GCN network updates node features to obtain M^l as

$$M^l = F_M(M^{l-1}, P^{l-1}). \tag{5}$$

After obtaining M^l, the graph embeddings are updated using Equation (1). The function F_M represents the update function for node features, following the propagation mode of the base GCN network, and is modeled as

$$M^l = \sigma(w^{l-1} M^{l-1} \Gamma^l). \tag{6}$$

Here, Γ^l represents the learnable parameter matrix of layer l, and σ denotes the activation function. $w^{l-1} \in \mathbb{R}^{N \times N}$ is the regularization aggregation weight matrix for all nodes in layer $l-1$, composed of weights $w_i^{l-1}, i \in N$, for individual nodes, modeled as

$$\begin{aligned} d^{l-1} &= \widetilde{D}^{-\frac{1}{2}} \widetilde{A} \widetilde{D}^{-\frac{1}{2}}, \\ w_i^{l-1}(R) &= \text{softmax}(\| d_i^{l-1}(R) - P^{l-1}(R) \|_\beta), \\ R &= \{i\} \cup N_i, \quad 1 \le i \le N. \end{aligned} \tag{7}$$

Here, $d^{l-1} \in \mathbb{R}^{N \times N}$ is a normalized local aggregation weight matrix, computed from the graph's adjacency matrix $\widetilde{A}$ and degree matrix $\widetilde{D}$ [50]. N represents the number of sample contracts. $d_i^{l-1}(R) \in \mathbb{R}^{|N_i|+1}$ represents the local aggregation weight of node i in layer $l-1$. The aggregation scope R includes node i and its neighboring nodes N_i. $P^{l-1}(R)$ represents the significance of node i at layer $l-1$, obtained by aggregating the significance of nodes within the same scope.

Node significance reflects the importance of nodes based on aggregation results, while local aggregation weights emphasize the structural importance of nodes. We focus on inconsistencies in node importance under these two criteria. When a node is structurally crucial but insignificant in results, or vice versa, this conflict reveals areas requiring further attention. A significant difference between the two criteria suggests greater node importance, indicating that the node plays a significant role in at

Authorized licensed use limited to: Consortium - Saudi Arabia SDL. Downloaded on February 23,2026 at 18:39:25 UTC from IEEE Xplore. Restrictions apply.

least one perspective. Therefore, the regularization aggregation weight w_i^{l-1} for node i is determined based on the distance between local aggregation weights and significance, with β as a parameter used to control the distance norm.

In summary, the three components—node characteristic distillation, node significance evaluation, and network regularization—achieve mutually reinforcing, self-circulating updates. Graph Neural Memory dynamically aggregates and extracts graph embeddings from node features. Ultimately, the neural network obtains T^{L+1} as the final embedding representation of the graph, where L is the number of GCN layers. In our work, the CFG and the ASG are input into this neural network, resulting in two graph embedding representations, T_{cfg} and T_{asg}. The embedding for contract i is modeled as

$$T_{SC_i} = T_{cfg_i} || T_{asg_i}. \tag{8}$$

where "||" denotes the concatenation operation.

It is worth noting that for contracts without publicly available source code and only deployed bytecode, we generate the embedding representation T_{cfg} solely based on their CFGs. In this case, the final embedding of contract i is modeled as:

$$T_{SC_i} = T_{cfg_i}. \tag{9}$$

E. Smart Contract Vulnerability Detection

A Multilayer Perceptron model, F_{NMLP}, is used to predict the labels of contracts based on T_{SCi} as

$$P(lab_i) = \text{softmax}(F_{\text{NMLP}}(T_{SC_i})). \tag{10}$$

Here, $lab_i^v \in lab_i$, where $lab_i^v = 0$ indicates that contract i has no vulnerability v, and $lab_i^v = 1$ indicates that contract i has vulnerability v. When $v = 0$, it determines whether contract i is negative. $P(lab_i^v)$ represents the predicted probability that contract i has label v. In a multi-class problem, v has nine possible values, while in a binary classification problem, v has two values. The model F_{NMLP} consists of two fully connected layers with ReLU activation functions, with a dropout rate of 0.2.

The loss function used for optimization is cross-entropy loss is

$$L = -\sum_{i=1}^{N} \log(p(lab_i^{y_i})). \tag{11}$$

Here, y_i represents the true label of contract i.

V. Experiments and Evaluation

This section presents experiments conducted to evaluate the performance and efficiency of our tool. Section V-A covers the experimental setup, including environmental configuration, network parameters, and dataset composition. Section V-B provides a performance comparison between our tool and five competitors. Section V-C evaluates our proposal's capability to detect eight types of vulnerabilities simultaneously. Finally, Section V-D analyzes the experimental results.

TABLE II
Details of Competing Tools (including Slither)

Analyzer	Input	Method	Release Year	Cite Count	Star/Fork
SmartCheck 2.0 [30]	S	SA	2019	721	354/88
DR_GCN [24]	S	ML	2020	229	132/24
ConFuzzius 0.0.2 [33]	S	SA	2022	69	78/22
DLVA [11]	B	ML	2023	2	1/2
Mythril 0.24.8 [17]	S	SA	2024	177	3,735/703
Slither [32]	S	SA	2024	511	5,045/918
BGVD	B	ML	2024	NA	NA
BSGVD	S+B	ML	2024	NA	NA

A. Experimental Setup

1) Computer Setup: The experiments were conducted on a cloud server with 60GB of memory, an RTX 3090 GPU (24GB VRAM), and an AMD EPYC 9754 128-core Processor. The server ran Ubuntu 22.04, with TensorFlow 2.13.1 and Python 3.10.8 as the software environment.

2) Network Hyperparameters: The graph neural network had a learning rate of 0.001, 100 epochs, and a dropout rate of 0.05. The following hyperparameters were also tuned using the grid search method:

- Number of layers: {2, 3, 4}
- Neuron size: {128, 256, 512}
- Cross MLP layers: {1, 2}
- Activation function: {ReLU, LeakyReLU, GeLU}
- Distance norm parameter β: {1, 2, 3}

The dataset was split into training, testing, and validation sets. Various combinations of hyperparameters were tested to train models on the training and testing sets separately. The combination that yielded the best performance was selected. The model was then retrained using the combined training and testing sets as the new training set, and the final results were evaluated on the validation set.

3) Dataset: There is no standardized benchmark dataset for smart contract vulnerabilities, and existing detection tools offer limited labeled datasets. These datasets are insufficient for model training. Therefore, we evaluated multiple vulnerability detection tools and concluded that Slither is one of the best available. Slither is actively maintained, with high recognition and strong detection performance (details about Slither can be found in Table II). Earlier Table I showed the specific types of vulnerabilities and the number of contracts in our dataset, all labeled with Slither.

B. Performance Evaluation

1) Selection of Competing Tools: Several factors were considered in selecting competing tools. First, only tools capable of detecting Ethereum smart contracts were selected.

Authorized licensed use limited to: Consortium - Saudi Arabia SDL. Downloaded on February 23,2026 at 18:39:25 UTC from IEEE Xplore. Restrictions apply.

Second, the tools had to include open-source core code to ensure reproducibility. Additionally, the tools' detection scope should overlap with our approach to allow a meaningful comparison.

Additionally, there were two other goals in selecting competing tools. First, we aimed to include solutions capable of identifying vulnerabilities using both bytecode and source code analysis. However, classic tools like Oyente and Osiris, developed in the early stages, can only handle contracts written in older versions of Solidity. With rapid updates in EVM and Solidity versions, these tools cover only a small portion of contracts in our dataset. Finally, after filtering the available detection tools, only DLVA remained as a candidate that utilized contract bytecode as input.

Second, we aimed to compare both static analysis and machine learning solutions. Most existing solutions are based on static analysis strategies such as fuzzing and symbolic execution. Many machine learning-based solutions lack open-source code or do not provide the core processing steps, making them unsuitable for comparison experiments (e.g., AME [22] and TMP [24]).

In the end, we selected five smart contract vulnerability detection tools for comparison experiments, as summarized in Table II. These include two machine learning-based solutions and three static analysis-based solutions. Table II also provides information on the versions of each tool, their input formats, detection methods, and the release dates of their latest versions. In the "Input" column, "S" and "B" represent the requirement for the contract's source code and bytecode, respectively. And "S + B" means that both are required. In the "Method" column, "SA" refers to static analysis, while "ML" represents machine learning methods. Additionally, the table includes citation counts on Google Scholar and the number of stars and forks on GitHub for each corresponding project. These indicators allow us to roughly assess the advancement, recognition, and popularity of these tools.

2) Performance Comparison Results: The final experimental results are presented in Table III. The first column lists the types of vulnerabilities targeted for detection. Not all competing tools are capable of detecting all eight types of vulnerabilities. Therefore, vulnerabilities are organized as rows, and the accuracy, recall, precision, and F1 score of each tool for detectable vulnerabilities are presented as performance metrics. All experiments used ten-fold cross-validation.

In addition, the proportion of detectable contracts was included as a metric to evaluate the tools' ability to perform reliable detection. Detection failures were often caused by abnormal raises or excessive execution times. It is worth mentioning that during tool selection in Section V-B.1, we had already excluded tools with too few detectable contracts. Contracts that failed detection were excluded from the calculation of other detection metrics.

Across all eight vulnerability types, our tool achieves the highest accuracy in seven and consistently maintains the highest F1 score. Notably, it significantly outperforms others in detecting vulnerabilities such as controlled-array-length, unchecked-transfer, erc20-interface, and constant-function-asm.

Fig. 3 presents comparative graphs for individual metrics (accuracy and F1 score). For each vulnerability, we visually compare the performance of the best competitor against our BSGVD and BGVD.

Compared to the best-performing competitor, our tool shows significant accuracy improvements ranging from 26.9% to 80.83% for the aforementioned four vulnerabilities, along with F1 score improvements ranging from 35.62% to 78.96%. For the remaining three vulnerabilities, our tool shows accuracy improvements ranging from 3.26% to 7.37%, and F1 score improvements of 18.42% to 38.25%. Our accuracy is 4.03% lower than the highest competitor (DLVA) for the locked-ether vulnerability. However, our tool achieves a 16.86% higher F1 score and a 29.81% higher precision. This indicates that while our accuracy is slightly lower, our true positive rate significantly exceeds that of DLVA. In summary, our detection model achieves high accuracy while balancing precision and recall effectively, ensuring a high true positive rate.

Among the eight types of vulnerabilities, reentrancy is the only type detectable by all tools. Fig. 4 illustrates the performance of each tool. It is evident that our proposed tool achieves excellent performance.

Confuzzius shows high precision but low accuracy and recall, indicating a high true positive rate but failure to identify all positive instances. This highlights the limitations of fuzz testing methods, whose operations are overly randomized, making it difficult to cover all execution paths and reduce the false negative rate.

Mythril faces similar challenges. As a symbolic execution method, it can apply strict rules to filter out vulnerable contracts, but the difficulty in precisely defining vulnerabilities results in low prediction accuracy and precision. SmartCheck exhibits high accuracy and recall, which we attribute to its broader definition of vulnerabilities, leading to a high recall rate but also a high false positive rate. The performance of DLVA is notable, likely due to its training set, which consists of small-sized contracts (the authors did not disclose if the model was trained on larger contracts). DR_GCN performs effectively as a machine learning method, ranking second only to our tools. The strong performance of our tool can be attributed to the capabilities of the graph neural network to precisely extract critical local graph structures, resulting in more accurate overall graph embeddings.

Although we primarily focus on contracts with source code deployed on the blockchain, our tool also shows high performance when considering only bytecode. BSGVD, which uses both bytecode and source code, outperforms BGVD in detecting most vulnerabilities. BGVD shows a slight decrease in accuracy ranging from 0.3% to 3.04%, which we consider acceptable.

3) Efficiency Comparison Results: Machine learning methods have a significant advantage over traditional static analysis methods regarding execution speed. In Fig. 5, machine learning methods' detection time includes preprocessing raw data (e.g., contract bytecode or source code). Our tool runs in less than 1 second, significantly faster than static analysis tools, and performs comparably to DLVA, which also employs machine learning methods. Earlier Fig. 4 showed that DR_GCN ranks

Authorized licensed use limited to: Consortium - Saudi Arabia SDL. Downloaded on February 23,2026 at 18:39:25 UTC from IEEE Xplore. Restrictions apply.

TABLE III
RESULTS OF VULNERABILITY DETECTION PERFORMANCE COMPARISON

Vulnerability	Detection Tool	Finished Contracts(%)	Accuracy(%)	Precision(%)	Recall(%)	F1(%)
controlled-delegatecall	ConFuzzius	7.32	12.28	46.67	12.28	21.20
	DLVA	100	29.31	6.63	29.31	10.81
	Mythril	100	9.60	44.12	9.60	15.77
	SmartCheck	100	92.54	43.53	92.54	59.21
	BGVD	100	98.43	95.43	95.11	95.28
	BSGVD	100	99.13	99.26	95.71	97.46
controlled-array-length	DLVA	100	67.60	27.14	67.60	38.73
	SmartCheck	100	48.58	31.73	48.58	38.39
	BGVD	100	94.50	91.23	92.81	91.98
	BSGVD	100	94.45	91.44	92.31	91.86
unchecked-transfer	DLVA	100	15.09	21.42	15.09	17.71
	SmartCheck	100	6.98	31.73	6.98	11.44
	BGVD	100	95.05	95.54	96.14	96.04
	BSGVD	100	95.92	96.33	97.01	96.67
erc20-interface	DLVA	99.25	10.04	14.57	10.04	11.89
	SmartCheck	100	51.61	76.68	51.61	61.70
	BGVD	100	95.34	95.65	96.76	96.20
	BSGVD	100	96.72	97.01	97.62	97.32
locked-ether	ConFuzzius	86.26	71.01	99.80	71.01	83.08
	DLVA	100	98.18	65.51	98.18	78.58
	SmartCheck	100	32.11	69.60	32.11	43.95
	BGVD	100	91.11	93.27	92.96	93.11
	BSGVD	100	94.15	95.32	95.54	95.42
shadowing-abstract	SmartCheck	100	91.23	59.51	91.23	72.03
	BGVD	100	92.62	88.94	85.28	86.94
	BSGVD	100	94.49	90.55	90.37	90.45
constant-function-asm	DLVA	100	3.84	1.46	3.84	2.12
	SmartCheck	100	54.26	43.50	54.26	48.29
	BGVD	100	96.40	93.32	93.05	93.17
	BSGVD	100	96.70	94.33	92.93	93.61
reentrancy-eth	ConFuzzius	32.41	25.23	91.21	25.23	39.53
	DLVA	100	7.00	2.02	7.00	3.14
	DR_GCN	100	87.07	86.43	84.13	84.72
	Mythril	100	7.09	43.11	7.09	12.18
	SmartCheck	100	87.29	48.68	87.29	62.50
	BGVD	100	93.98	85.78	86.67	86.14
	BSGVD	100	94.66	87.52	87.81	87.62

second to our tools in performance, however its time cost is almost three times that of ours.

The detection time of Mythril is notably longer. During our experiments, we constrained its execution time based on the parameters recommended by the authors. The results indicate that the time cost for static detection is significantly higher compared to machine learning methods, which aligns with expectations. Although DR_GCN also employs machine learning techniques, its execution time is prolonged. This is because DR_GCN assigns types to nodes and edges in the contract graph and involves operations that delete and modify nodes and edges during subsequent processing. Due to these complex operations and the need for numerous parameters, its efficiency is affected. The slightly slower performance of our tool compared to DLVA is due to the need for creating an additional graph structure. However, by reducing node and edge features, we avoided the efficiency issues.

C. Detecting All 8 Vulnerabilities Simultaneously

After concluding the discussion on individually detecting each vulnerability, we now evaluate our tool's performance in detecting all eight vulnerabilities using a single model.

Tables IV and V present the experimental results. Notably, BSGVD achieves an accuracy of 91.26% and an F1 score of 97.42%. Additionally, BGVD achieves a respectable accuracy of 87.34% and an F1 score of 96.43%. These results demonstrate the effectiveness of our approach.

Authorized licensed use limited to: Consortium - Saudi Arabia SDL. Downloaded on February 23,2026 at 18:39:25 UTC from IEEE Xplore. Restrictions apply.

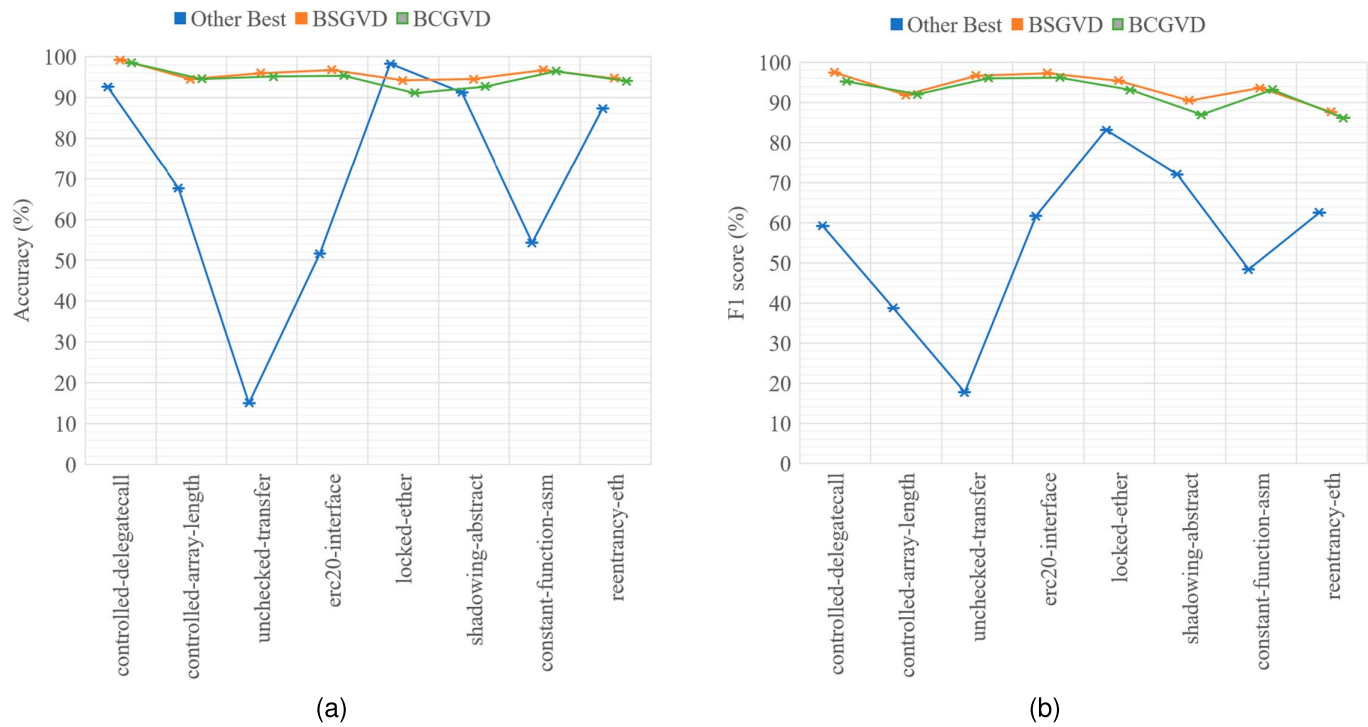

Fig. 3. Comparison of individual metrics. (a) Accuracy, (b) F1 score.

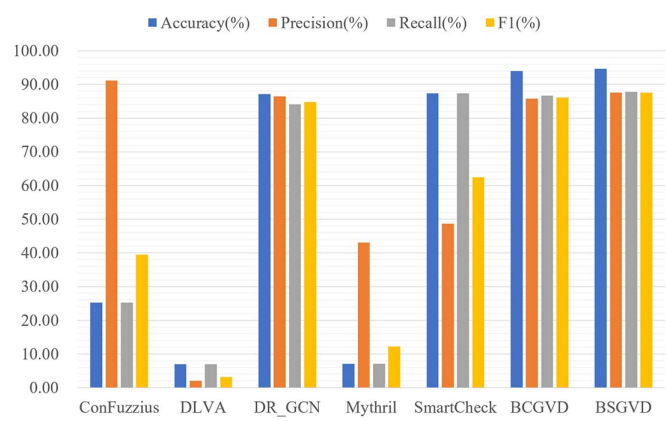

Fig. 4. Performance of each tool in detecting the "reentrancy-eth" vulnerability.

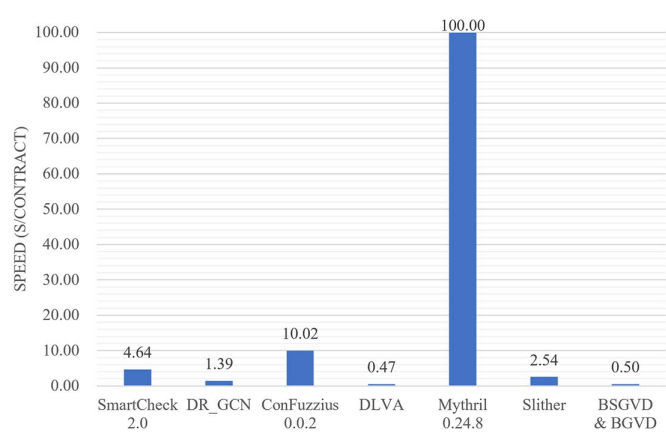

Fig. 5. Results of vulnerability detection efficiency comparison.

The accuracy decreases when simultaneously detecting eight vulnerabilities compared to individual detection due to correlations among different vulnerabilities, which may share similar code or data structures. This requires the model to learn both commonalities and distinctions among vulnerabilities, increasing learning complexity. Additionally, the imbalanced dataset used in our study adds to the prediction difficulty. Nevertheless, the model's final performance remains relatively high, demonstrating the robustness of our approach.

D. Discussion

The experiments demonstrate that our tool performs exceptionally well, achieving high accuracy and F1 scores while maintaining fast detection speeds. It exhibits strong competitive performance compared to other smart contract vulnerability detection tools. This section provides a more in-depth discussion and analysis of the experimental results.

1) Satisfactory Spatial Cost: The numerous parameters in machine learning algorithms and the abundance of features in datasets pose significant challenges. CFG and ASG of smart contract code contain a large number of nodes and edges, but not all edges have the same significance. For instance, CFG edges can be classified into conditional and unconditional jumps. The edges in ASG represent even more enriched relationships, including data flow, control flow, and data structure dependencies.

Initially, we aimed to consider all these factors to generate the graph structure, ensuring semantic integrity. However, storing intermediate data representations and loading the dataset posed significant challenges to device memory. Additionally, managing edge features required more parameters, further complicating the model. Thus, we opted to omit all edge features.

Authorized licensed use limited to: Consortium - Saudi Arabia SDL. Downloaded on February 23,2026 at 18:39:25 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
RESULTS OF JOINT DETECTION OF 8 VULNERABILITIES BY BSGVD

Vulnerability	Accuracy	Precision	Recall	F1
controlled-delegatecall	96.15	-	-	-
controlled-array-length	94.74	-	-	-
unchecked-transfer	92.32	-	-	-
erc20-interface	96.95	-	-	-
locked-ether	94.59	-	-	-
shadowing-abstract	92.67	-	-	-
constant-function-asm	83.46	-	-	-
reentrancy-eth	79.21	-	-	-
total	91.26	97.44	97.39	97.42

TABLE V
RESULTS OF JOINT DETECTION OF 8 VULNERABILITIES BY BGVD

Vulnerability	Accuracy	Precision	Recall	F1
controlled-delegatecall	90.71	-	-	-
controlled-array-length	82.75	-	-	-
unchecked-transfer	91.39	-	-	-
erc20-interface	94.83	-	-	-
locked-ether	90.33	-	-	-
shadowing-abstract	83.03	-	-	-
constant-function-asm	71.85	-	-	-
reentrancy-eth	76.50	-	-	-
total	87.34	96.22	96.66	96.43

While this could theoretically reduce detection performance, it significantly optimized spatial cost and improved time efficiency without compromising overall robustness.

2) Capability to Exclusively Detect Bytecode: Most contracts on the blockchain are not deployed with source code, making it important to detect vulnerabilities solely from bytecode. Some existing methods address this challenge using techniques such as knowledge distillation [16]. As a byproduct, we developed BGVD, which uses only bytecode and still demonstrates strong detection performance, which is undoubtedly encouraging.

3) Broad Coverage of Vulnerabilities: One significant advantage of machine learning algorithms is that they do not require manually defined rules, achieving automated training and detection based on labeled datasets. Our tool demonstrates high detection performance for eight vulnerabilities, covering various triggering factors, which suggests a degree of universality. In the future, we can apply this tool to detect other vulnerabilities by simply adding specific datasets, thereby expanding its capabilities.

VI. CONCLUSION

We presented a novel method for detecting vulnerabilities in Ethereum smart contracts, targeting eight of the most common vulnerabilities. Our approach provides accurate and efficient detection using either the contract's source code or bytecode, making it applicable to almost all smart contracts. In addition to binary classification for individual vulnerabilities, we implemented multi-class detection to identify all eight vulnerabilities simultaneously. Our tool was guided by Slither and trained on a labeled dataset of actual Ethereum smart contracts. It consistently achieved detection accuracies above 94% for all eight vulnerabilities, with a peak accuracy of 99.13%. Furthermore, it maintained high F1 scores, significantly outperforming existing tools. For multi-class detection, our tool achieved an accuracy of 91.26% and an F1 score of 97.42%. These results demonstrate our approach's robustness, effectiveness, and competitive edge in addressing the vulnerabilities present in smart contracts.

REFERENCES

[1] CoinMarketCap, "Ethereum (Eth) price, charts, market cap, and other metrics," CoinMarketCap, 2024. Accessed: May 31, 2024. [Online]. Available: https://coinmarketcap.com/currencies/ethereum

[2] Z. Zheng, S. Xie, H. Dai, X. Chen, and H. Wang, "An overview of blockchain technology: Architecture, consensus, and future trends," in *Proc. IEEE Int. Congr. Big Data, BigData Congr.*, G. Karypis and J. Zhang, Eds., Honolulu, HI, USA: IEEE Computer. Soc., Jun. 2017, pp. 557–564.

[3] W. Metcalfe et al., "Ethereum, smart contracts, DAPPS," in *Blockchain and Crypt Currency*, vol. 77, 2020, pp. 77–93.

[4] M. Buchwald, "Smart contract dispute resolution: The inescapable flaws of blockchain-based arbitration," *Univ. Pennsylvania Law Rev.*, vol. 168, 2019, Art. no. 1369.

[5] E. Lai and W. Luo, "Static analysis of integer overflow of smart contracts in ethereum," in *Proc. 4th Int. Conf. Cryptography, Secur. Privacy (ICCSP)*, Nanjing, China, New York, NY, USA: ACM, Jan. 2020, pp. 110–115.

[6] C. Liu, H. Liu, Z. Cao, Z. Chen, B. Chen, and B. Roscoe, "Reguard: Finding reentrancy bugs in smart contracts," in *Proc. 40th Int. Conf. Softw. Eng., Companion (ICSE)*, Gothenburg, Sweden, M. Chaudron, I. Crnkovic, M. Chechik, and M. Harman, Eds., New York, NY, USA: ACM, 2018, pp. 65–68.

[7] J. Krupp and C. Rossow, "Teether: Gnawing at ethereum to automatically exploit smart contracts," in *Proc. 27th USENIX Secur. Symp., USENIX Secur.*, W. Enck and A. P. Felt, Eds., Baltimore, MD, USA: USENIX Assoc., 2018, pp. 1317–1333.

[8] S. Sayeed, H. Marco-Gisbert, and T. Caira, "Smart contract: Attacks and protections," *IEEE Access*, vol. 8, pp. 24416–24427, 2020.

[9] M. I. Mehar et al., "Understanding a revolutionary and flawed grand experiment in blockchain: The DAO attack," *J. Cases Inf. Technol.*, vol. 21, no. 1, pp. 19–32, 2019.

[10] S. Palladino, "The parity wallet hack explained," Santiago Palladino, 2017. Accessed: Jul. 19, 2017. [Online]. Available: https://blog.openzeppelin.com/on-the-parity-wallet-multisig-hack-405a8c12e8f7

[11] T. Abdelaziz and A. Hobor, "Smart learning to find dumb contracts," in *Proc. 32nd USENIX Secur. Symp., USENIX Secur.*, Anaheim, CA, USA, J. A. Calandrino and C. Troncoso, Eds., USENIX Assoc., 2023, pp. 1775–1792.

[12] L. Luu, D.-H. Chu, H. Olickel, P. Saxena, and A. Hobor, "Making smart contracts smarter," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Vienna, Austria, E.R. Weippl, S. Katzenbeisser, C. Kruegel, A.C. Myers, and S. Halevi, Eds., New York, NY, USA: ACM, Oct. 2016, pp. 254–269.

[13] W. Zou et al., "Smart contract development: Challenges and opportunities," *IEEE Trans. Softw. Eng.*, vol. 47, no. 10, pp. 2084–2106, Oct. 2021.

[14] Z. Liao, Z. Zheng, X. Chen, and Y. Nan, "Smartdagger: A bytecode-based static analysis approach for detecting cross-contract vulnerability," in *Proc. 31st ACM SIGSOFT Int. Symp. Softw. Testing Anal.*, Virtual Event, South Korea, S. Ryu and Y. Smaragdakis, Eds., New York, NY, USA: ACM, 2022, pp. 752–764.

[15] J. Huang et al., "Hunting vulnerable smart contracts via graph embedding based bytecode matching," *IEEE Trans. Inf. Forensics Security*, vol. 16, pp. 2144–2156, 2021.

[16] P. Qian, Z. Liu, Y. Yin, and Q. He, "Cross-modality mutual learning for enhancing smart contract vulnerability detection on bytecode," in *Proc. ACM Web Conf. (WWW)*, Austin, TX, USA, Y. Ding, J. Tang,

Authorized licensed use limited to: Consortium - Saudi Arabia SDL. Downloaded on February 23,2026 at 18:39:25 UTC from IEEE Xplore. Restrictions apply.

J. F. Sequeda, L. Aroyo, C. Castillo, and G.-J. Houben, Eds., May 2023, pp. 2220–2229. ACM,

[17] Bernhard Mueller, *A Framework for Bug Hunting on the Ethereum Blockchain.* 2017. Available: https://github.com/ConsenSys/mythril

[18] C. F. Torres, J. Schütte, and R. State, "Osiris: Hunting for integer bugs in ethereum smart contracts," in *Proc. 34th Annu. Comput. Secur. Appl. Conf. (ACSAC)*, San Juan, PR, USA, New York, NY, USA: ACM, 2018, pp. 664–676.

[19] B. Jiang, Y. Liu, and W. K. Chan, "Contractfuzzer: Fuzzing smart contracts for vulnerability detection," in *Proc. 33rd ACM/IEEE Int. Conf. Automated Softw. Eng. (ASE)*, Montpellier, France, M. Huchard, C. Kästner, and G. Fraser, Eds., New York, NY, USA: ACM, 2018, pp. 259–269.

[20] J. Xu, F. Dang, X. Ding, and M. Zhou, "A survey on vulnerability detection tools of smart contract bytecode," in *Proc. IEEE 3rd Int. Conf. Inf. Syst. Comput. Aided Educ. (ICISCAE)*, Piscataway, NJ, USA: IEEE, 2020, pp. 94–98.

[21] S. Hwang, S. Choi, J.-M. Shin, and Y.-H. Choi, "Codenet: Code-targeted convolutional neural network architecture for smart contract vulnerability detection," *IEEE Access*, vol. 10, pp. 32595–32607, 2022.

[22] Z. Liu, P. Qian, X. Wang, L. Zhu, Q. He, and S. Ji, "Smart contract vulnerability detection: From pure neural network to interpretable graph feature and expert pattern fusion," in *Proc. 13th Int. Joint Conf. Artif. Intell. (IJCAI)*, Virtual Event, Montreal, QC, Canada, Z.-H. Zhou, Ed., 2021, pp. 2751–2759.

[23] P. Qian, Z. Liu, Q. He, R. Zimmermann, and X. Wang, "Towards automated reentrancy detection for smart contracts based on sequential models," *IEEE Access*, vol. 8, pp. 19685–19695, 2020.

[24] Y. Zhuang, Z. Liu, P. Qian, Q. Liu, X. Wang, and Q. He, "Smart contract vulnerability detection using graph neural network," in *Proc. 29th Int. Joint Conf. Artif. Intell. (IJCAI)*, C. Bessiere, Ed., 2020, pp. 3283–3290.

[25] X. Yu, H. Zhao, B. Hou, Z. Ying, and B. Wu, "Deescvhunter: A deep learning-based framework for smart contract vulnerability detection," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, Shenzhen, China, Piscataway, NJ, USA: IEEE, 2021, pp. 1–8.

[26] X. Sun, L. Tu, J. Zhang, J. Cai, B. Li, and Y. Wang, "*ASSBert*: Active and semi-supervised bert for smart contract vulnerability detection," *J. Inf. Secur. Appl.*, vol. 73, Mar. 2023, Art. no. 103423.

[27] P. Tsankov, A. M. Dan, D. Drachsler-Cohen, A. Gervais, F. Bünzli, and M. T. Vechev, "Securify: Practical security analysis of smart contracts," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, Toronto, ON, Canada, D. Lie, M. Mannan, M. Backes, and X. Wang, Eds., New York, NY, USA: ACM, 2018, pp. 67–82.

[28] M. Rodler, W. Li, G. O. Karame, and L. Davi, "Sereum: Protecting existing smart contracts against re-entrancy attacks," in *Proc. 26th Annu. Netw. Distrib. Syst. Secur. Symp. (NDSS)*, San Diego, CA, USA: The Internet Soc., Feb. 2019, p. 2019.

[29] C. Schneidewind, I. Grishchenko, M. Scherer, and M. Maffei, "Ethor: Practical and provably sound static analysis of ethereum smart contracts," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, Virtual Event, USA, J. Ligatti, X. Ou, J. Katz, and G. Vigna, Eds., New York, NY, USA: ACM, 2020, pp. 621–640.

[30] S. Tikhomirov, E. Voskresenskaya, I. Ivanitskiy, R. Takhaviev, E. Marchenko, and Y. Alexandrov, "Smartcheck: Static analysis of ethereum smart contracts," in *Proc. 1st IEEE/ACM Int. Workshop Emerg. Trends Softw. Eng. Blockchain (WETSEB@ICSE)*, Gothenburg, Sweden, R. Tonelli, G. Destefanis, S. Counsell, and M. Marchesi, Eds., New York, NY, USA: ACM, 2018, pp. 9–16.

[31] P. Qian, Z. Liu, Q. He, B. Huang, D. Tian, and X. Wang, "Smart contract vulnerability detection technique: A survey," 2022, *arXiv:2209.05872*.

[32] J. Feist, G. Grieco, and A. Groce, "Slither: A static analysis framework for smart contracts," 2019, *arXiv:1908.09878*.

[33] C. F.Torres, A. K.Iannillo, A. Gervais, and R. State, "Confuzzius: A data dependency-aware hybrid fuzzer for smart contracts," in *Proc. IEEE Eur. Symp. Secur. Privacy (EuroSP)*, Vienna, Austria, Piscataway, NJ, USA: IEEE, 2021, pp. 103–119.

[34] X. Yi, Y. Fang, D. Wu, and L. Jiang, "Blockscope: Detecting and investigating propagated vulnerabilities in forked blockchain projects," in *Proc. 30th Annu. Netw. Distrib. Syst. Secur. Symp. (NDSS)*, San Diego, CA, USA: The Internet Soc., 2023, pp. 1–16.

[35] X. Gu, H. Zhang, and S. Kim, "Deep code search," in *Proc. 40th Int. Conf. Softw. Eng.*, 2018, pp. 933–944.

[36] Y. Sun et al., "Gptscan: Detecting logic vulnerabilities in smart contracts by combining GPT with program analysis," in *Proc. 46th IEEE/ACM Int. Conf. Softw. Eng. (ICSE)*, Lisbon, Portugal, New York, NY, USA: ACM, 2024, pp. 166:1–166:13.

[37] P. Momeni, Y. Wang, and R. Samavi, "Machine learning model for smart contracts security analysis," in *Proc. 17th Int. Conf. Privacy, Secur. Trust (PST)*, Fredericton, NB, Canada, Piscataway, NJ, USA: IEEE, Aug. 2019, pp. 1–6.

[38] J.-W. Liao, T.-T. Tsai, C.-K. He, and C.-W. Tien, "Soliaudit: Smart contract vulnerability assessment based on machine learning and fuzz testing," in *Proc. 6th Int. Conf. Internet Things, Syst., Manage. Secur. (IOTSMS)*, Granada, Spain, M. A. Alsmirat and Y. Jararweh, Eds., Piscataway, NJ, USA: IEEE, 2019, pp. 458–465.

[39] Z. Liu, P. Qian, X. Wang, Y. Zhuang, L. Qiu, and X. Wang, "Combining graph neural networks with expert knowledge for smart contract vulnerability detection," *IEEE Trans. Knowl. Data Eng*, vol. 35, no. 2, pp. 1296–1310, Feb. 2023.

[40] E. Surya Negara, A. Nizar Hidayanto, R. Andryani, and R. Syaputra, "Survey of smart contract framework and its application," *Information*, vol. 12, no. 7, p. 257, 2021.

[41] B. K. Mohanta, S. S. Panda, and D. Jena, "An overview of smart contract and use cases in blockchain technology," in *Proc. 9th Int. Conf. Comput., Commun. Netw. Technol. (ICCCNT)*, Bengaluru, India, Piscataway, NJ, USA: IEEE, 2018, pp. 1–4.

[42] F. E.Allen, "Control flow analysis," in *Proc. Symp. Compiler Optim.*, Urbana-Champaign, IL, USA, Robert S. Northcote, Ed., New York, NY, USA: ACM, 1970, pp. 1–19.

[43] A. Orailoglu and D. Gajski, "Flow graph representation," in *Proc. 23rd ACM/IEEE Des. Automat. Conf.*, Las Vegas, NV, USA, D. Thomas, Ed., Los Alamitos, CA, USA: IEEE Comput. Soc. Press, Jun. 1986, pp. 503–509.

[44] F. Josselin, "A tool for recovering a control flow graph (CFG) from EVM bytecode," 2018. Accessed: Dec. 8, 2018. [Online]. Available: https://github.com/crytic/evm_cfg_builder

[45] J. Zhang, X. Wang, H. Zhang, H. Sun, K. Wang, and X. Liu, "A novel neural source code representation based on abstract syntax tree," in *Proc. 41st Int. Conf. Softw. Eng. (ICSE)*, Montreal, QC, Canada, J. M. Atlee, T. Bultan, and J. Whittle, Eds., New York, NY, USA: ACM, 2019, pp. 783–794.

[46] Y. Zhang and D. Liu, "Toward vulnerability detection for ethereum smart contracts using graph-matching network," *Future Internet*, vol. 14, no. 11, p. 326, 2022.

[47] S. Singh Kushwaha, S. Joshi, D. Singh, M. Kaur, and H.-N. Lee, "Systematic review of security vulnerabilities in ethereum blockchain smart contract," *IEEE Access*, vol. 10, pp. 6605–6621, 2022.

[48] P. Bojanowski, E. Grave, A. Joulin, and T. Mikolov, "Enriching word vectors with subword information," *Trans. Assoc. Comput. Linguistics*, vol. 5, pp. 135–146, Jun. 2017.

[49] W. Pei et al., "Saliency-aware regularized graph neural network," *Artif. Intell.*, vol. 328, Mar. 2024, Art. no. 104078.

[50] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *Proc. 5th Int. Conf. Learn. Representations (ICLR) Conf. Track Proc.*, Toulon, France, 2017, pp. 1–14.

Chang Xu received the Ph.D. degree in computer science from Beihang University, Beijing, China, in 2013. Currently, she is an Associate Professor with the School of Cyberspace Science and Technology, Beijing Institute of Technology, Beijing. Her research interests include security and privacy in VANET, and big data security.

Huaiyu Xu received the bachelor's degree in computer science and technology from Beijing Institute of Technology, Beijing, China, in 2023. She is currently working toward the master's degree with the School of Cyberspace Science and Technology, Beijing Institute of Technology, Beijing. Her research interests include security and privacy in artificial intelligence, and blockchain applications.

Authorized licensed use limited to: Consortium - Saudi Arabia SDL. Downloaded on February 23,2026 at 18:39:25 UTC from IEEE Xplore. Restrictions apply.

Liehuang Zhu (Senior Member, IEEE) received the Ph.D. degree in computer science from Beijing Institute of Technology, Beijing, China, in 2004. Currently, he is a Professor with the School of Cyberspace Science and Technology, Beijing Institute of Technology, Beijing. His research interests include security protocol analysis and design, group key exchange protocols, wireless sensor networks, and cloud computing.

Xiaodong Shen received the bachelor's and master's degrees from the School of Computer Science and Technology, Beijing Institute of Technology, Beijing, China, in 2015 and 2018, respectively. He is currently working toward the Ph.D. degree with the School of Cyberspace Science and Technology, Beijing Institute of Technology, Beijing. His research interests include crowdsensing, security and privacy in IoT, and blockchain applications.

Kashif Sharif (Senior Member, IEEE) received the M.S. degree in information technology and the Ph.D. degree in computing and informatics from The University of North Carolina at Charlotte, USA, in 2004 and 2012, respectively. Currently, he is an Associate Professor with Beijing Institute of Technology, Beijing, China. His research interests include information centric networks, blockchain distributed ledger technologies, wireless and sensor networks, software defined networks, and data center networking. He also serves as an Associate Editor of IEEE ACCESS.

Authorized licensed use limited to: Consortium - Saudi Arabia SDL. Downloaded on February 23,2026 at 18:39:25 UTC from IEEE Xplore. Restrictions apply.